

快三系列客户端连线服务端开发说明

1. 连线方式

- 透过 websocket 连线 (IP, Port, Path)
- Client与Server双方以 protobuf 格式传输参数或资料
- 参数资料结构：(maincmd-subcmd, proto message payload)

2. 串接流程

main cmd / sub cmd 说明

- 7 : MDM_LOGON (登入)
 - 7 : SUB_LOGON_REQ__LOGIN (登入)
 - 106 : SUB_LOGON_RESP__SUCCESS (登入成功)
 - 107 : SUB_LOGON_RESP__LOGIN_ERROR (登入异常)
- 8 : MDM_USER (用户)
 - 7 : SUB_USER_REQ__SIT (入房坐下)
- 500 : MDM_GAME (游戏)
 - GameReqCode : client -> server 的请求代码
 - GameResCode : server -> client 的回应代码

GameReqCode 说明

```
public interface GameReqCode {  
    // 下注单 -> Req.BetReq  
    int C2S_BET = 1030;  
    // 查询投注记录 -> Req.ListBetDetailLogReq  
    int C2S_LIST_BET_DETAIL_LOG = 1031;  
    // 查询开奖纪录 -> Req.ListRoundInfo  
    int C2S_LIST_ROUND_INFO = 1032;  
    // 查询全游戏的近n场投注记录 -> no payload  
    int C2S_LIST_GAME_LATEST_BETTING_ROUND_DETAIL_LOG = 1033;  
    // 切换GroupId, 不可切到DEFAULT_GROUPID("") -> Req.Group  
    int C2S_CHANGE_GROUP = 1034;  
    // 刷新金钱 -> no payload  
    int C2S_REFRESH_SCORE = 1035;  
    // 重置GroupId=DEFAULT_GROUPID(""), 可应用在回到直播间列表的情境 -> no payload  
    int C2S_RESET_GROUP = 1036;  
    // 盘面指定 -> Req.TestAssign  
    int C2S_TEST_ASSIGN = 1050;  
}
```

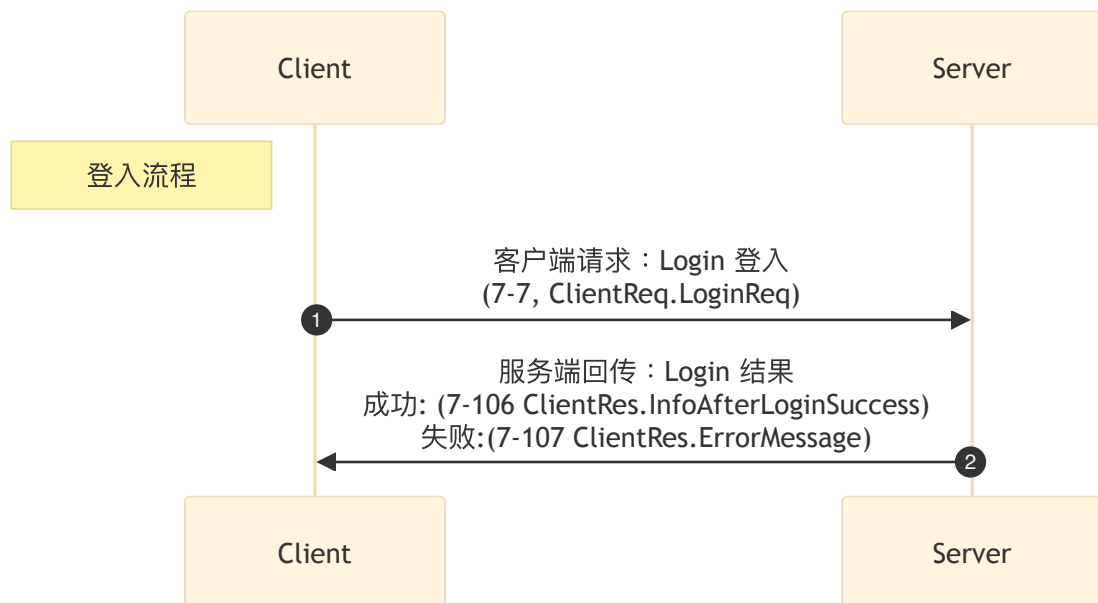
GameResCode 说明

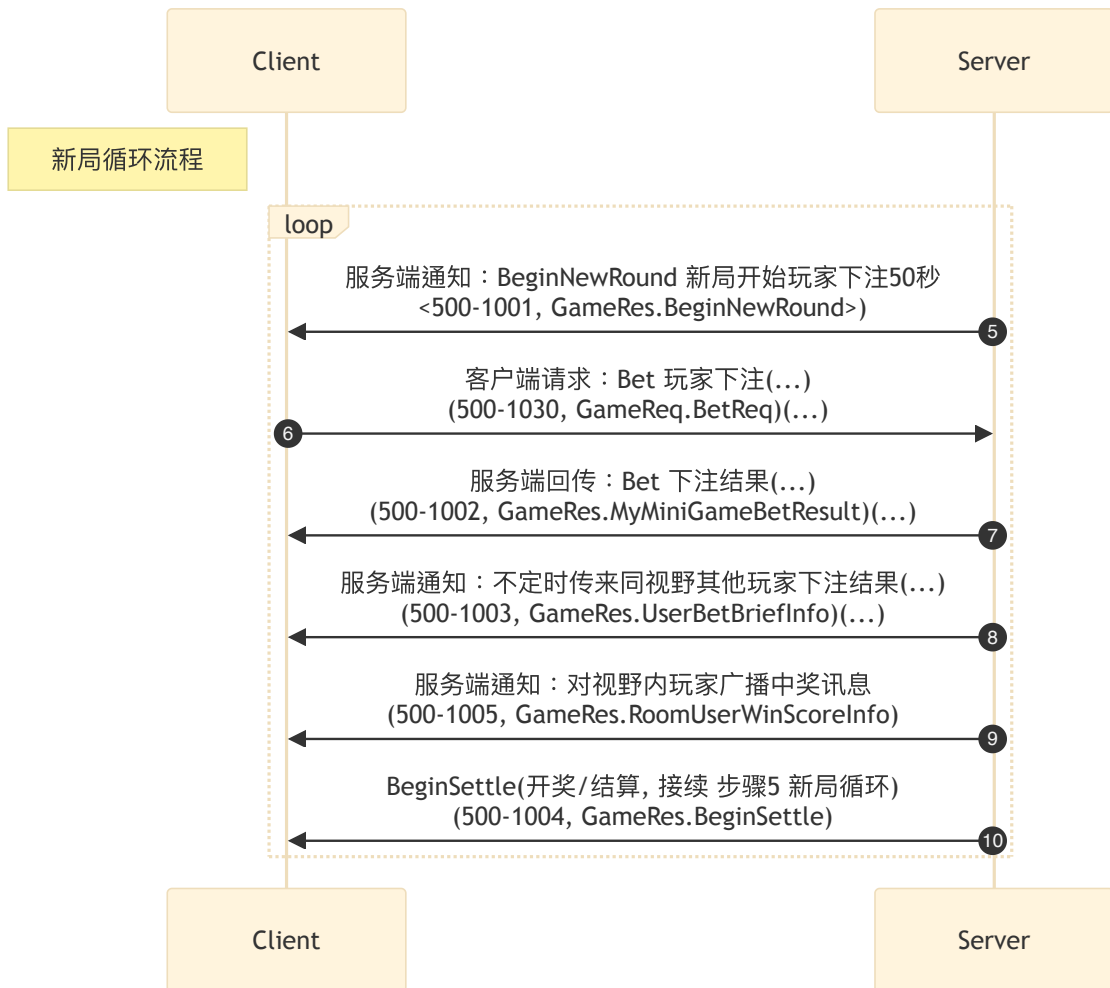
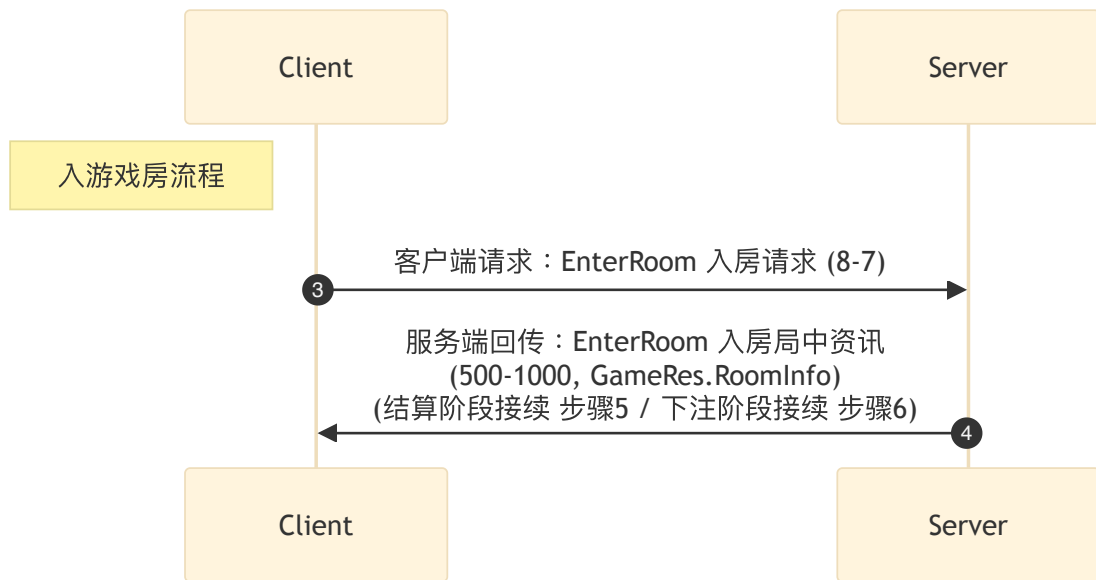
```

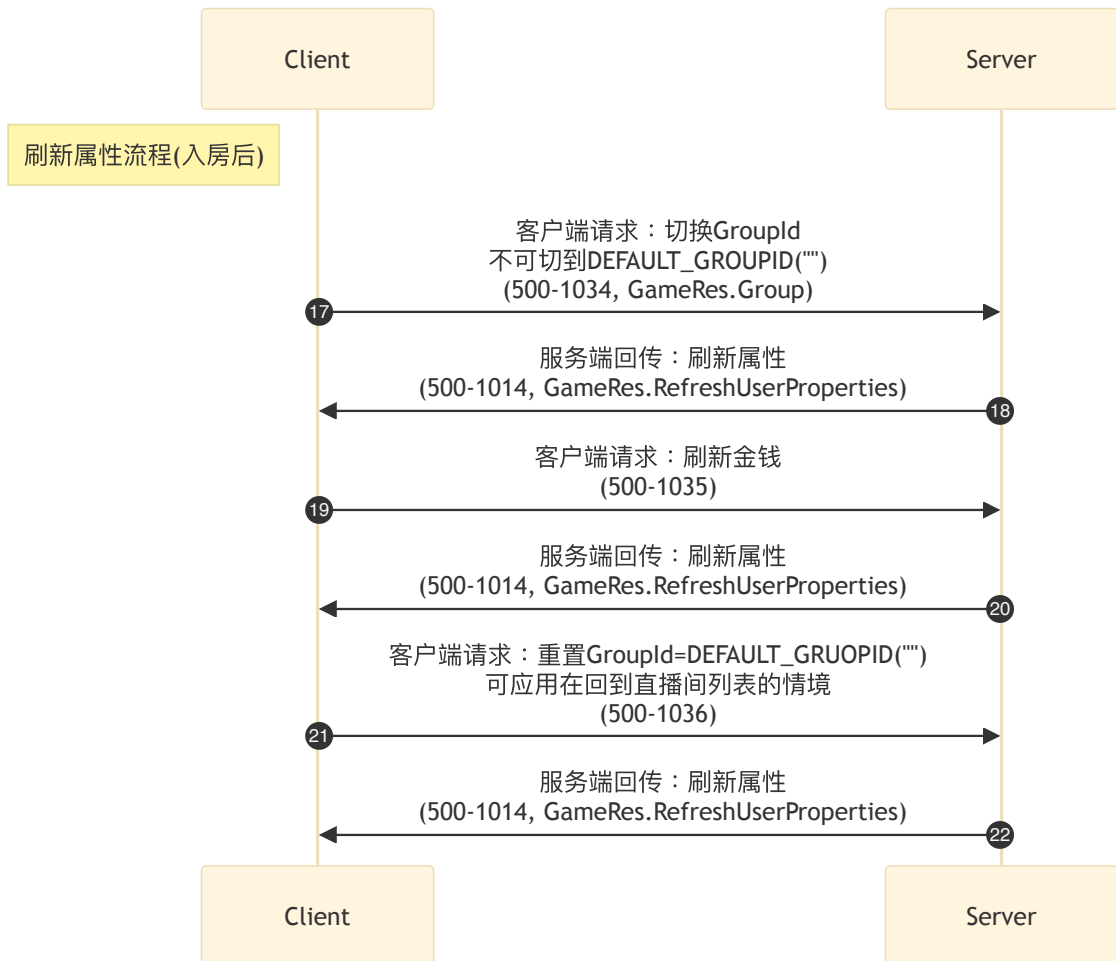
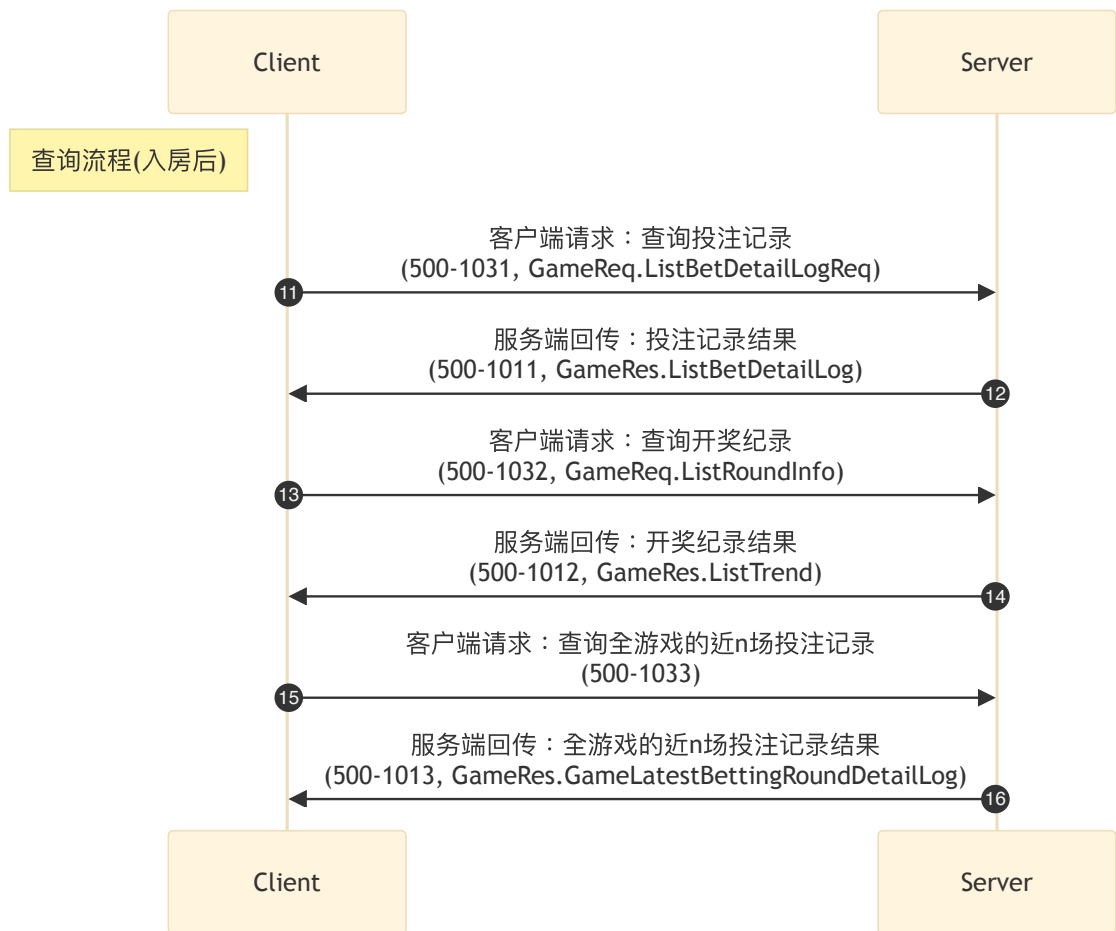
public interface GameResCode {
    // 进入房间，取得房间资讯 -> Res.RoomInfo
    int S2C_ENTER_ROOM_INFO = 1000;
    // 开始新局 -> Res.BeginNewRound
    int S2C_BEGIN_ROUND = 1001;
    // 玩家自己下注结果 -> Res.MyMiniGameBetResult
    int S2C_USER_BET_RESULT = 1002;
    // 同视野其他玩家下注 -> Res.UserBetBriefInfo
    int S2C_NOTIFY_OTHER_BET = 1003;
    // 进入结算 -> Res.BeginSettle
    int S2C_BEGIN_SETTLE = 1004;
    // 广播其同视野玩家中奖资讯(含自己) -> Res.RoomUserWinScoreInfo
    int S2C_NOTIFY_USER_WINSORE = 1005;
    // 查询详细投注记录结果 -> Res.ListBetDetailLog
    int S2C_LIST_BET_DETAIL_LOG_RESULT = 1011;
    // 查询开奖历史纪录 -> Res.ListTrend
    int S2C_LIST_TREND_RESULT = 1012;
    // 查询最近有下注的n期详细投注记录 -> GameLatestBettingRoundDetailLog
    int S2C_LIST_GAME_LATEST_BETTING_ROUND_DETAIL_LOG = 1013;
    // 刷新属性 -> RefreshUserProperties
    int S2C_REFRESH_USER_PROPS = 1014;
    // 更新游戏资讯 -> Res.MiniGameInfo
    int S2C_UPDATE_MINIGAME_INFO = 1015;
}

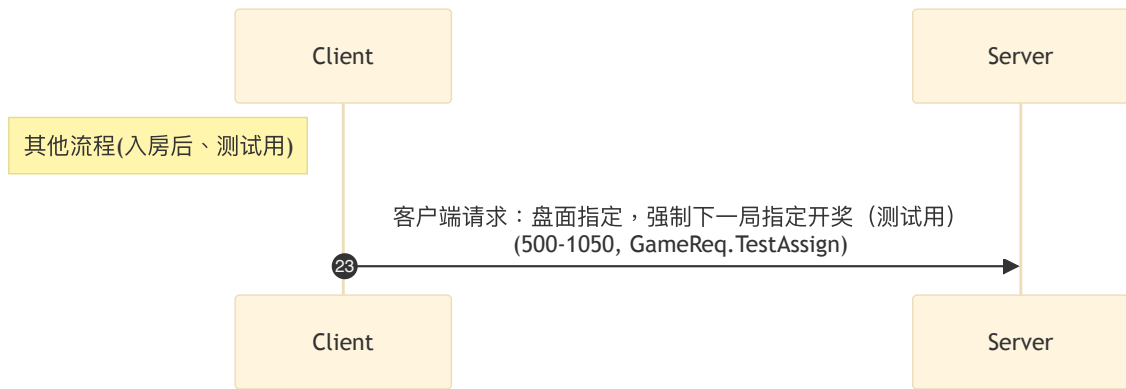
```

各项流程交握协议









3. proto message payload 资料结构

ClientReq 资料结构 (Client -> Server)

7-7：登录游戏服

```

message LoginReq{
    string agent_name = 1;
    string token = 2; // 登录token。client从大厅/商户登录获取
    string version = 3; // client 版本

    string base = 4; // 热更版本。仅供 cocos 连 logon 使用，其他不用传
    string game_version = 5; // 小游戏版本。仅供 cocos 连 logon 使用，其他不用传

    int32 platform = 6; // 账号平台 2=2n1（默认）、4=试玩、5=h5。cocos 可不传
    int32 server = 7; // serverID。游戏服合并端口后，cocos 用此字段识别进哪个服
    int32 requestId = 8; // 用于识别重复请求的唯一id。仅供 cocos 识别登录重发使用，h5 不用传
    string nickname = 9; // 昵称
}

```

ClientRes 资料结构 (Server -> Client)

7-106：登录成功

```

message InfoAfterLoginSuccess {
    string resource_base_url = 1; // 更新资源的root路径
}

```

7-107：登录异常

```

message ErrorMessage {
    sint32 code = 1; // 异常错误码
    string desc = 2; // 提示文案
    int32 server_id = 7; // 跳转到服务器：服务器Id
    string ip = 3; // 跳转到服务器：服务器ip
    int32 port = 4; // 跳转到服务器：服务器端口
    int32 server_type = 5; // 跳转到服务器：服务器游戏类型
}

```

```

    int32 sub_type = 6; // 跳转到服务器：服务器房间类型
}

// 携带数据 ErrorMessage
//     code = 0 一般错误
//     desc = (错误或异常说明)
//     desc = ...
//     code = 200 其他情况
//     code = 1001 token验证失败
//     code = 1005 清除客户端游戏服缓存
//     code = 1007 商户信息无效
//     code = 1008 商户api调用异常
//     预留 code = 1009 失败：预留

```

GameReq 资料结构 (Client -> Server)

各种单位定义：

```

// 机率 1=0.0001, ex: 1234 = 12.34%
// 金钱 1=0.01, ex: 123 = 1.23元
// 赔率 1=1, ex: 1.2 = 1.2倍
// 时间 1=1ms, ex: 12400 = 12.4 s = 12400 ms

```

下注

```

message BetReq {
    // 小游戏 id
    int32 miniGameId = 1;
    // 下注资料
    repeated AreaBet areaBet = 2;
}

```

每个注区的押注资料

```

message AreaBet {
    // 注区ID:
    int32 areaId = 1;
    // 玩家注区总下注金额
    int32 betScore = 2;
}

```

----- 投注与开奖纪录相关 -----

查询投注记录

```

message ListBetDetailLogReq {
    int32 miniGameId = 1;
    int32 status = 2; // [0=全部, 1=已中奖, 2=未中奖], 未成立的单不列入投注记录
    int32 yyyy = 3; // 查询日期-年
    int32 mm = 4; // 查询日期-月
    int32 dd = 5; // 查询日期-日
    int64 beginDataId = 6; // 从哪个Id往前找, singular, 0=最新的区段, ex: beginDataId = 100, 找
}

```

查询开奖纪录

```
message ListRoundInfo {
    int32 miniGameId = 1;
    int64 beginDataId = 2; // 从哪个索引往前找, singular, 0=最新的区段
}
```

视野

```
message Group {
    string id = 1; // group id
    string data = 2; // 透传资料 (转抛)
}
```

----- 测试用相关 -----

盘面指定

```
message TestAssign {
    // 小游戏 id
    int32 miniGameId = 1;
    // 牌面
    repeated int32 elements = 2;
}
```

GameRes 资料结构 (Server -> Client)

----- 登入相关 -----

登入后给玩家资料

```
message UserData {
    int64 userId = 1; // 用户id
    int64 score = 2; // 身上的钱
}
```

----- 进房后相关 -----

进房后给, 房间资讯

```
message RoomInfo {
    // 当前所处阶段: 1. 下注 2. 结算
    int32 stage = 1;
    // 当前阶段剩余时间倒计时(ms)
    int32 countDown = 2;
    // 小游戏资讯
    repeated MiniGameInfo miniGameInfoList = 3;
    // 我的未开奖投注资讯(所有miniGame, 给介面上此期末开奖)
```

```

    repeated BetBriefInfo uncheckedBetBriefInfoList = 4;
    // 玩家资料
    UserData self = 5; // 暂时放置 EnterRoom，以后会再移走。
    // 小游戏显示顺序
    repeated int32 miniGameShowOrder = 6;
    // 玩家登入时间
    int64 userLoginTime = 7;
}

```

小游戏资讯

```

message MiniGameInfo {
    // 小游戏 id
    int32 id = 1;
    // 注区资讯
    repeated AreaInfo areaInfoList = 2;
    // 当前期号
    string roundId = 3;
    // 开奖纪录(若进场时为结算阶段，则包含此局结果)
    Trend trend = 4;
}

```

注区资讯

```

message AreaInfo {
    int32 id = 1; // 注区ID
    string name = 2; // 注区名称
    int32 type = 3; // 注区类别
    string typeName = 4; // 注区类别名称
    float odds = 5; // 赔率
    int32 minLimit = 6; // 投注下限
    int32 maxLimit = 7; // 投注上限
}

```

开奖资讯（通用格式定义，能符合未来新游戏的扩充）

```

message RoundInfo {
    // 资料索引
    int64 dataId = 1;
    // 期号
    string roundId = 2;
    // 牌面种类
    int32 elementType = 3; // 1:扑克牌, 2:麻将牌, 3:天九牌, 4:骰子...
    // 表演资讯
    repeated ActorPerform performs = 4;
    // 获胜注区列表：(内容数值定义：注区ID)
    repeated int32 winAreaIds = 5;
}

message ActorPerform {
    // 牌面
    repeated int32 elements = 1; // (根据elementType决定)
    // 牌型
    repeated int32 patterns = 2; // (根据miniGameId决定) 牛牛, 铁支
    // 结果
}

```



```
    repeated int32 performResult = 3; // (通用) 0:undefined, 1:胜, 2:败, 3:和
}
```

开奖历史

```
message Trend {
    repeated RoundInfo roundInfoList = 1;
}
```

开始新一轮

```
message BeginNewRound {
    // 当前阶段剩余时间倒计时(ms)
    int32 countDown = 1;
    // 期号
    string roundId = 2;
}
```

投注资讯

```
message BetInfo {
    // 注区ID
    int32 areaId = 1;
    // 注区总下注金额
    int64 betScore = 2;
}
```

下注结果

```
message MyMiniGameBetResult{
    // 小游戏 id
    int32 miniGameId = 1;
    // 押注资讯
    repeated BetResultInfo betResultInfoList = 2;
    // 当前玩家剩余金币
    //int64 score = 3;
}

message BetResultInfo {
    BetInfo betInfo = 1;
    int32 result = 2;// OrderStatus
}
```

广播其他同房玩家下注资讯(不含自己)

```
message UserBetBriefInfo {
    // 玩家id
    int64 userId = 1;
    // 玩家昵称
    string userNickName = 2;
    // 押注资讯
}
```

```

    BetBriefInfo betBriefInfo = 3;
}

message BetBriefInfo{
    // 小游戏 id
    int32 miniGameId = 1;
    // 押注资讯
    repeated BetInfo betInfoList = 2;
}

```

各小游结算

```

message BeginSettle {
    // 当前阶段剩余时间倒计时(ms)
    int32 countDown = 1;
    // 各小游戏开奖结果
    repeated MiniGameSettle miniGameSettles = 2;
    // 各小游戏结算结果
    repeated ListBetDetailLog gameBetDetailLogs = 3;
    // 最终身上金额
    // int64 score = 4;
}

```

单一小游戏结算, 进入**settle**就会传开奖结果, 但开奖结果与新的投注记录须等**settle**倒数结束才可以呈现到画面

```

message MiniGameSettle {
    // 小游戏 id
    int32 miniGameId = 1;
    // 开奖资讯
    RoundInfo roundInfo = 2;
    // 此局结算后新增的投注纪录
    repeated BetDetailLog betLogs = 3;
}

```

查询最近有下注的n期详细投注记录

```

message GameLatestBettingRoundDetailLog{
    repeated ListBetDetailLog gameLogs = 1;
}

```

广播其同房玩家中奖资讯(含自己), 这一局的所有单总和，有赢就发

```

message RoomUserWinScoreInfo {
    repeated UserWinScoreInfo userWinInfoList = 1;
}

message UserWinScoreInfo {
    int64 userId = 1; // 玩家id
    string userNickName = 2; // 玩家暱称
    int32 miniGameId = 3; // 玩家中奖游戏
    int64 winScore = 4; // 中奖金额
}

```

选择性刷新玩家属性

```
message RefreshUserProperties {
    optional string groupId = 1;
    optional int64 score = 2;
}
```

----- 投注与开奖纪录相关 -----

```
message ListBetDetailLog {
    int32 miniGameId = 1;
    repeated BetDetailLog betDetailLogs = 2;
}
```

每一单的投注记录, 未成立的单不列入

```
message BetDetailLog {
    // 资料索引序号
    int64 dataId = 1;
    // 期号
    string roundId = 2;
    // 注区
    int32 areaId = 3;
    // 投注金额
    int64 betScore = 4;
    // 中奖金额(没中奖为0)
    int64 deltaScore = 5;
    // 投注时间
    int64 betTime = 6;
}

message ListTrend {
    int32 miniGameId = 1;
    repeated RoundInfo infoList = 2;
}
```

6. 连线问题排除

- 以 telnet 一类工具确认 IP, Port 监听是否正常

7. 串接注意事项

- token 之正确性
- AgentName 之正确性
- ServerID 之正确性